Phala Network TEE 실제 코드 구현 분석

1. 개요

이 문서는 **Phala Network의 TEE 컴포넌트들의 실제 소스 코드**를 분석하여 Phactory, Pink Runtime, SideVM의 구체적인 구현 세부사항을 다룹니다.

2. Phactory 실제 구현 분석

2.1 Phactory 라이브러리 구조

실제 모듈 구성

```
// crates/phactory/src/lib.rs에서 실제 구현
pub mod benchmark;
mod bin_api_service;
mod contract_result;
pub mod contracts;
mod cryptography;
mod im_helpers;
mod light_validation;
mod nts;
mod prpc_service;
mod secret_channel;
```

```rust
mod storage;
mod system;
mod types;

// 주요 타입 별칭
type RuntimeHasher = <chain::Runtime as frame_system::Config>::Hashing;
pub type PRuntimeLightValidation = LightValidation<chain::Runtime>;

// 핵심 익스포트
pub use chain::BlockNumber;
pub use contracts::pink;
pub use prpc_service::RpcService;
pub use storage::ChainStorage;
pub use system::gk;
pub use types::{BaseBlockInfo, BlockInfo};
```

실제 구현의 특징:

- **Light Validation**: 경량 블록체인 검증 시스템
- **Secret Channel**: 암호화된 통신 채널
- **NTS (Network Time Security)**: 시간 동기화 보안
- **Benchmark**: 성능 측정 시스템

2.2 시스템 모듈 실제 구현

트랜잭션 에러 타입

```rust
// crates/phactory/src/system/mod.rs의 실제 에러 정의
#[derive(Encode, Decode, Debug, Clone, thiserror::Error)]
#[error("TransactionError: {:?}", self)]
pub  enum TransactionError {
BadInput,
BadOrigin,
Other(String),
// 일반적인 에러
InsufficientBalance,
NoBalance,
UnknownError,
BadContractId,
BadCommand,
SymbolExist,
AssetIdNotFound,
NotAssetOwner,
BadSecret,
BadMachineId,
FailedToSign,
BadDecimal,
DestroyNotAllowed,
ChannelError,
```

```
// 게이트키퍼 관련
NotGatekeeper,
MasterKeyLeakage,
BadSenderSignature,
// ... 더 많은 에러 타입들
}
```

```
pub  type TransactionResult = Result<Option<ExecSideEffects>, TransactionError>;
```

합의 버전 관리

```
// 실제 합의 버전 상수
pub(crate) const MAX_SUPPORTED_CONSENSUS_VERSION: u32 = 0;
```

실제 구현에서 발견한 점:

- **세밀한 에러 분류**: 50개 이상의 구체적인 에러 타입
- **사이드 이펙트 추적**: `ExecSideEffects` 로 실행 결과 추적
- **합의 버전 제한**: 현재 버전 0만 지원

3. Pink Runtime 실제 구현 분석

3.1 Pink Runtime 아키텍처

실제 클러스터 구조

```
// crates/phactory/src/contracts/pink.rs의 실제 구현
```

```rust
#[derive(Serialize, Deserialize, Default, Clone, ::scale_info::TypeInfo)]
pub struct ClusterConfig {
pub log_handler: Option<AccountId>,
pub runtime_version: (u32, u32),
pub secret_salt: [u8; 32],
#[serde(default)]
pub js_runtime: Option<Hash>,
}

#[derive(Serialize, Deserialize, Clone, ::scale_info::TypeInfo)]
pub struct Cluster {
pub id: ContractClusterId,
pub config: ClusterConfig,
pub storage: ClusterStorage,
}
```

실제 구현의 특징:

- **JavaScript 런타임 지원**: `js_runtime` 필드로 JS 실행 환경 관리
- **로그 핸들러**: 전용 로그 처리 계정 지정 가능
- **시크릿 솔트**: 클러스터별 고유 암호화 솔트

런타임 핸들 시스템

```rust
// 실제 런타임 핸들 구현
```

```
pub struct RuntimeHandleMut<'a> {
cluster: &'a mut Cluster,
logger: Option<CommandSender>,
pub(crate) effects: Option<ExecSideEffects>,
}


impl RuntimeHandleMut<'_> {
fn readonly(&self) -> RuntimeHandle {
RuntimeHandle {
cluster: self.cluster,
logger: self.logger.clone(),
}
}
fn execute_mut<T>(&mut self, f: impl FnOnce((u32, u32)) -> T) -> T {
let ver = self.readonly().runtime_version();
using_ocalls(self, || f(ver))
}


pub fn call(
&mut self,
contract: AccountId,
input_data: Vec<u8>,
mode: ExecutionMode,
```

```
tx_args: TransactionArguments,
) -> Vec<u8> {
context::using_entry(contract.clone(), tx_args.origin.clone(), move || {
self.contract_call(contract, input_data, mode, tx_args)
})
}


pub fn instantiate(
&mut self,
code_hash: Hash,
instantiate_data: Vec<u8>,
salt: Vec<u8>,
mode: ExecutionMode,
tx_args: TransactionArguments,
) -> Vec<u8> {
// 컨트랙트 ID 생성
let buf = phala_types::contract::contract_id_preimage(
tx_args.origin.as_ref(),
code_hash.as_ref(),
self.readonly().cluster_id().as_ref(),
&salt,
);
let entry = AccountId::from(blake2_256(&buf));
```

```
context::using_entry(entry, tx_args.origin.clone(), move || {
self.contract_instantiate(code_hash, instantiate_data, salt, mode, tx_args)
})
}
}
```

실제 구현의 핵심:

- **Mutable/Immutable 분리**: 읽기 전용과 쓰기 가능 핸들 분리
- **컨텍스트 관리**: `using_entry` 로 실행 컨텍스트 설정
- **버전 기반 실행**: 런타임 버전에 따른 조건부 실행

3.2 Pink Runtime 라이브러리 구조

실제 라이브러리 설계

```
// crates/pink/runtime/src/lib.rs의 실제 구현
//! # Pink Runtime Library
//!
//! This crate contains the core functionality of the Pink runtime, which powers
//! It is compiled into dynamic libraries and included with the `pruntime` binar
//! there are several versions of `libpink.so`. The `pruntime` would choose the
//! when it starts, based on instructions from the Phala blockchain.

extern crate alloc;
```

```rust
pub mod capi;
mod contract;
mod runtime;
mod storage;
pub mod types;

pub use crate::contract::{ContractExecResult, ContractInstantiateResult, Contr

/// Returns a tuple containing the major and minor version numbers of the curren
pub fn version() -> (u32, u32) {
let major = env!("CARGO_PKG_VERSION_MAJOR")
.parse()
.expect("Invalid major version");

let minor = env!("CARGO_PKG_VERSION_MINOR")
.parse()
.expect("Invalid minor version");

(major, minor)
}
```

실제 구현의 특징:

- **동적 라이브러리**: `libpink.so` 로 컴파일되어 런타임에 로드

- **버전 관리**: 메이저/마이너 버전으로 호환성 관리
- **C API**: `capi` 모듈로 C 인터페이스 제공
- **no_std 지원**: `extern crate alloc` 로 제한된 환경 지원

4. SideVM 실제 구현 분석

4.1 SideVM 서비스 시스템

실제 명령어 타입

```
// crates/sidevm/host-runtime/src/service.rs의 실제 구현
pub enum Command {
// SideVM 인스턴스 중지
Stop,
// SideVM 메시지 전송
PushMessage(Vec<u8>),
// 시스템 메시지 전송
PushSystemMessage(SystemMessage),
// RPC 쿼리 전송
PushQuery {
origin: Option<AccountId>,
payload: Vec<u8>,
reply_tx: OneshotSender<Vec<u8>>,
},
// 태스크 스케줄링 가중치 업데이트
```

```
UpdateWeight(u32),
// HTTP 요청 처리
HttpRequest(IncomingHttpRequest),
}
```

실제 종료 이유 타입

```
#[derive(Debug, Clone, Copy, Serialize, Deserialize, derive_more::Display)]
pub  enum ExitReason {
/// 프로그램이 main에서 정상 반환
Exited(i32),
/// 외부 Stop 명령으로 중지
Stopped,
/// 입력 채널이 닫힘 (Stop 명령으로 인한)
InputClosed,
/// 프로그램 패닉
Panicked,
/// 태스크 future가 드롭됨 (Stop 명령으로 인한)
Cancelled,
/// 가스 체크로 인한 종료
OcallAborted(OcallAborted),
/// 이전 실행 인스턴스가 체크포인트에서 복구됨
Restore,
```

```
/// 코드 없이 배포되어 커스텀 코드 업로드 대기 중
WaitingForCode,
/// 코드가 너무 큼
CodeTooLarge,
/// 인스턴스 시작 실패
FailedToStart,
}
```

실제 서비스 런타임 구현

```
pub fn service(
worker_threads: usize,
out_tx: crate::OutgoingRequestChannel,
) -> (ServiceRun, Spawner) {
let worker_threads = worker_threads.max(1);
let runtime = tokio::runtime::Builder::new_multi_thread()
.max_blocking_threads(16)
// 추가 2개 스레드 이유:
// 하나는 블로킹 리액터 스레드용, 다른 하나는 Pink 시스템으로부터
// 채널 메시지 수신용
.worker_threads(worker_threads + 2)
.enable_all()
.build()
.unwrap();
```

```
let runtime_handle = runtime.handle().clone();
let (report_tx, report_rx) = channel(100);
let run = ServiceRun { runtime, report_rx };
let spawner = Spawner {
runtime_handle,
report_tx,
out_tx,
scheduler: TaskScheduler::new(worker_threads as _),
};
(run, spawner)
}
```

실제 구현의 특징:

- **정교한 스레드 관리**: 워커 스레드 + 2개 추가 스레드
- **태스크 스케줄링**: `TaskScheduler` 로 VM 인스턴스 스케줄링
- **비동기 처리**: Tokio 런타임 기반 완전 비동기 처리
- **세밀한 종료 이유**: 10가지 이상의 구체적인 종료 원인 추적

4.2 HTTP 요청 처리

실제 HTTP 요청 구조

```
pub struct IncomingHttpRequest {
pub(crate) head: HttpHead,
pub(crate) body_stream: DuplexStream,
```

```
pub(crate) response_tx: OneshotSender<anyhow::Result<HttpResponseHead>>,

}
```

실제 구현의 특징:

- **스트리밍 지원**: `DuplexStream` 으로 양방향 스트리밍
- **비동기 응답**: `OneshotSender` 로 비동기 응답 처리
- **헤드/바디 분리**: HTTP 헤더와 바디 스트림 분리 처리

5. 실제 구현에서 발견한 고급 기능들

5.1 JavaScript 런타임 통합

Pink에서 JavaScript 지원

```
// ClusterConfig에서 JavaScript 런타임 지원
pub  js_runtime: Option<Hash>,


// pink::chain_extension에서 JavaScript 관련 타입
use pink::chain_extension::{JsCode, JsValue};
```

발견한 점:

- Pink Runtime에서 **JavaScript 실행 지원**
- 클러스터별로 다른 JS 런타임 버전 사용 가능
- Chain Extension을 통한 JS 코드 실행

5.2 고급 메모리 관리

로컬 캐시 시스템

```
use pink_loader::{
local_cache::{self, StorageQuotaExceeded},
// ...
};
```

발견한 점:

- **스토리지 쿼터 시스템**: 메모리 사용량 제한
- **로컬 캐시**: 성능 최적화를 위한 캐싱 레이어
- **쿼터 초과 처리**: `StorageQuotaExceeded` 에러 타입

5.3 고급 보안 기능

SGX Quote 처리

```
use sgx_attestation::SgxQuote;
use once_cell::sync::Lazy;
use std::sync::Mutex;
```

발견한 점:

- **SGX Quote 직접 처리**: 저수준 SGX 증명 처리
- **스레드 안전성**: `Mutex` 와 `Lazy` 로 동시성 제어
- **Attestation 통합**: SGX 증명과 Pink Runtime 통합

5.4 성능 모니터링

메트릭 시스템

```
pub use sidevm_env::messages::{Metric, SystemMessage};
```

```
use sidevm::{
service::{Command as SidevmCommand, CommandSender, Metric, SystemMessage},
// ...
};
```

발견한 점:

- **실시간 메트릭**: SideVM 실행 메트릭 수집
- **시스템 메시지**: 시스템 상태 모니터링
- **성능 추적**: 실행 시간, 메모리 사용량 등 추적

6. 실제 코드와 추정 코드 비교

6.1 정확했던 추정들

- ✅ TEE 내 컨트랙트 실행 환경
- ✅ 클러스터 기반 워커 관리
- ✅ HTTP API 지원
- ✅ 비동기 처리 아키텍처

6.2 실제와 달랐던 부분들

- ❌ JavaScript 런타임 지원 (예상하지 못함)

- ❌ 동적 라이브러리 로딩 (`libpink.so`)
- ❌ 세밀한 종료 이유 분류 (10가지 이상)
- ❌ 스토리지 쿼터 시스템

6.3 새로 발견한 고급 기능들

- 🆕 **Pink Runtime 버전 관리**: 동적 라이브러리 버전 선택
- 🆕 **JavaScript 체인 익스텐션**: JS 코드 실행 지원
- 🆕 **고급 스케줄링**: TaskScheduler를 통한 정교한 리소스 관리
- 🆕 **실시간 메트릭**: 상세한 성능 모니터링
- 🆕 **스트리밍 HTTP**: 양방향 HTTP 스트리밍 지원

7. 결론

실제 TEE 코드 분석을 통해 Phala Network의 TEE 구현이 **매우 정교하고 고도로 최적화**되어 있음을 확인했습니다:

7.1 아키텍처 우수성

- **계층화된 설계**: Phactory → Pink Runtime → SideVM의 명확한 계층
- **모듈화**: 각 컴포넌트의 독립적 개발 및 업그레이드 가능
- **확장성**: JavaScript 런타임, 동적 라이브러리 로딩 등

7.2 보안 중심 설계

- **다층 검증**: SGX Quote부터 애플리케이션 레벨까지
- **격리 실행**: SideVM을 통한 추가적인 보안 경계
- **메모리 보안**: 쿼터 시스템, 안전한 할당/해제

7.3 성능 최적화

- **비동기 처리**: Tokio 기반 완전 비동기 아키텍처
- **스마트 스케줄링**: TaskScheduler를 통한 리소스 최적화
- **캐싱 시스템**: 다층 캐시로 성능 향상

7.4 개발자 친화성

- **풍부한 에러 타입**: 50개 이상의 구체적인 에러 분류
- **상세한 메트릭**: 실시간 성능 모니터링
- **유연한 실행 모드**: 다양한 실행 환경 지원

이러한 실제 구현 분석을 통해 Phala Network가 단순한 TEE 활용을 넘어 **프로덕션 급 TEE 컴퓨팅 플랫폼**을 구축했음을 알 수 있습니다.